

Computational Linear Algebra for Large Scale Problems

Homework 3: Image Compression Using SVD

Famà Daniele
s345153

Perlo Alessandro
s337131

A.Y. 2024/25

Abstract

In this optional homework we implement Singular Value Decomposition and test our implementation against an image compression task. Then, we compare image compression using SVD with the gold standard of image compression, which is the Discrete Cosine Transform based algorithm adopted by the JPEG lossy compression format. All the functions are in MATLAB code.

1 SVD Implementation

Given a matrix $A \in \mathbb{R}^{m \times n}$ ($m \geq n$, otherwise the matrix is transposed), the implementation of SVD $A = U\Sigma V^T$ we followed consists in the following steps.

1. The matrix A is bidiagonalized by means of two orthogonal matrices H, P such that $A = P^T B H^T$, where B is an upper bidiagonal matrix.
2. The singular value decomposition of $B = U_1 \Sigma V_1^T$ is computed using an optimized method for bidiagonal matrices.
3. The left singular vectors and right singular vectors of A are computed as follows.

$$U = P^T U_1, \quad V = H V_1$$

1.1 Bidiagonalization of a matrix

The bidiagonalization of a matrix is an essential step in the computation of the Singular Value Decomposition (SVD), particularly when applied to large-scale problems like image compression. This process simplifies the original matrix by transforming it into a bidiagonal form.

Bidiagonalization is particularly useful for significantly reducing the computational complexity of subsequent SVD operations, such as eigenvalue computation. The transformation to bidiagonal form is achieved through orthogonal transformations. In our case study, we implemented bidiagonalization using Householder reflections.

Given $A \in \mathbb{R}^{m,n}$ with $m \geq n$ the goal is to obtain:

$$B = \begin{pmatrix} \alpha_1 & \beta_1 & 0 & \dots & 0 \\ & \alpha_2 & \beta_2 & & \\ & & \ddots & \ddots & \vdots \\ & & & \ddots & \beta_{n-1} \\ & & & & \alpha_n \end{pmatrix}$$

The idea to obtain B matrix is the following. We consider:

$$A^{(0)} = A = \left(\begin{array}{c|c} & b_0^T \\ \hline a_0 & B_0 \end{array} \right)$$

Now, on this matrix, we apply the reflection $P^{(0)} = I_m - 2 \frac{a_0 a_0^T}{\|a_0\|_2^2}$:

$$P^{(0)} A^{(0)} = \left(\begin{array}{c|c} \alpha_1 & b_0^T \\ \hline 0 & \tilde{B}^{(0)} \\ \vdots & \\ \vdots & \\ 0 & \end{array} \right)$$

And now we apply again Householder from the right in order to vanish the term above the super-diagonal. To do this we multiply $P^{(0)} A^{(0)}$ with $H_0 = I_{n-1} - 2 \frac{b_0 b_0^T}{\|b_0\|_2^2}$:

$$H^{(0)} = \left(\begin{array}{c|c} I_1 & 0 \\ \hline 0 & \\ \vdots & \\ \vdots & H_0 \\ 0 & \end{array} \right)$$

The result is given by:

$$P^{(0)} A^{(0)} H^{(0)} = \left(\begin{array}{c|c} \alpha_1 & \beta_1 & 0 & \dots & 0 \\ \hline 0 & & & & \\ \vdots & & & & \\ \vdots & & & & B_1 \\ \vdots & & & & \\ 0 & & & & \end{array} \right)$$

In general P and H are symmetric block defined matrices:

$$P^{(k-1)} = \left(\begin{array}{c|c} I_{k-1} & \mathbf{0}^T \\ \hline 0 & \\ \vdots & \\ \vdots & P_{k-1} \\ 0 & \end{array} \right)$$

$$H^{(k-1)} = \left(\begin{array}{c|c} I_k & \mathbf{0}^T \\ \hline 0 & \\ \vdots & H_{k-1} \\ 0 & \end{array} \right)$$

where

$$P_{k-1} = I_{m-(k-1)} - 2 \frac{\tilde{a}_{k-1} \tilde{a}_{k-1}^T}{\|\tilde{a}_{k-1}\|_2^2}, H_{k-1} = I_{n-k} - 2 \frac{\tilde{b}_{k-1} \tilde{b}_{k-1}^T}{\|\tilde{b}_{k-1}\|_2^2}, A^{(k)} = P^{(k-1)} A^{(k-1)} H^{(k-1)} \quad (1)$$

Iteratively, we will multiply full $m \times m$ (P s) and $n \times n$ (H s) matrices. To reduce the computational cost, we compute the Householder vector ($u = x + \sigma e_1$ normalized) and apply it only by the not already bidiagonalized section of the original matrix. We end up with $B \in \mathbb{R}^{m,n}$ defined as $B = PAH$ with $P \in \mathbb{R}^{m,m} = P^{(n-1)} * \dots * P^{(0)}$, $A \in \mathbb{R}^{m,n}$ and $H \in \mathbb{R}^{m,m} = H^{(0)} * \dots * H^{(n-2)}$. Since each P and H are orthogonal, the products of each P_j and H_j are also orthogonal. Then A can be expressed as $A = P^T B H^T$.

1.2 SVD of a bidiagonal matrix

Once a bidiagonal matrix B is available, a QR-like algorithm for eigenvalues can be adopted to compute singular values of the matrix. Being B bidiagonal, its singular values are the square root of the eigenvalues of the tridiagonal matrix $B^T B$: we want to compute eigenvalues of this matrix. The algorithm consists in building a sequence of similar tridiagonal symmetric positive definite matrices T_i starting from $T_0 = B^T B$. Then, successive elements of the sequence are built as follows.

1. Apply the Cholesky factorization to $T_i = B_i^T B_i$.
2. Compute the next term of the sequence $T_{i+1} = B_i B_i^T$.
3. Check for convergence, namely that all elements of the codiagonal are below a given threshold.

Computing the Cholesky factorization and a matrix product at each iteration is costly. However, exploiting the structure of the matrices we deal with, we can compute elements of the matrices B_i instead. Let's denote:

- by $a_1, \dots, a_n$ the elements on the diagonal of B_i and by $b_1, \dots, b_{n-1}$ the elements on the upper co-diagonal of B_i ;
- by $\hat{a}_1, \dots, \hat{a}_n$ the elements on the diagonal of B_{i+1} and by $\hat{b}_1, \dots, \hat{b}_{n-1}$ the elements on the upper co-diagonal of B_{i+1} ;

then square of the entries of B_{i+1} can be computed from the square of entries of B_i as follows, for $j = 1, \dots, n-1$.

$$\begin{aligned} \hat{a}_1^2 &= a_1^2 + b_1^2 & \hat{a}_j^2 &= a_j^2 + b_j^2 - \hat{b}_{j-1}^2 & \hat{a}_n^2 &= a_n^2 + b_n^2 - \hat{b}_{n-1}^2 \\ \hat{b}_1^2 &= \frac{a_2^2 b_1^2}{\hat{a}_1^2} & \hat{b}_j^2 &= \frac{a_{j+1}^2 b_j^2}{\hat{a}_j^2} \end{aligned}$$

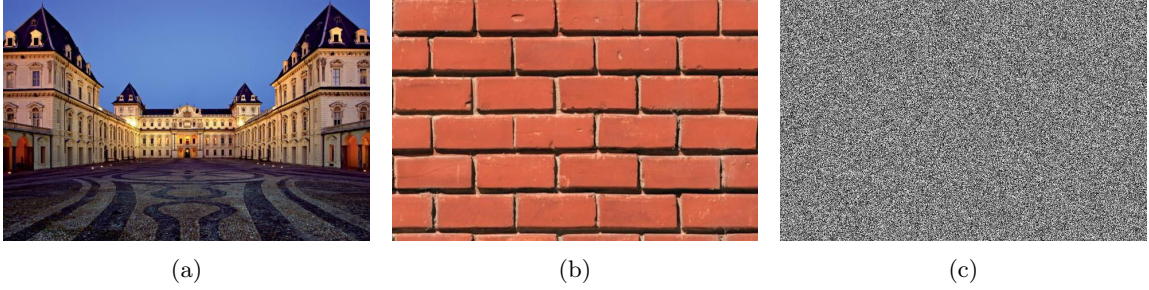


Figure 1: Original images of our case study.

At convergence, eigenvalues of $B^T B$ lie on the vector a consisting of the square of the diagonal elements of the matrix B_i . Eigenvectors, which are also right singular vectors of V , are later computed using shifted inverse power method as implemented in the function `shifted_inverse_power_method`. Being eigenvalues of $B^T B$ known with a certain degree of precision, convergence is very fast and takes few iterations. Left singular vectors are obtained from right singular vectors using the equation $Bv_i = \sigma_i u_i$.

2 Image Compression

Image compression plays a crucial role in reducing the amount of data required to store or transmit an image while maintaining its visual quality.

One powerful method for image compression is Singular Value Decomposition (SVD). When applied to an image, SVD permits us the representation of an image using less memory maintaining the most essential information. By performing SVD on an image, which is represented as a matrix, the image's data can be reduced by retaining only the largest singular values and vectors, which carry the most critical information. Let $A \in \mathbb{C}^{m,n}, m \geq n$, and let $A = U\Sigma V^T$ be its SVD. , then we can rewrite A as:

$$A = \sum_{i=1}^n \sigma_i u_i v_i^T \quad (2)$$

Intuitively, we can understand that large singular value σ_i corresponds to a large contribution of the term $u_i v_i^H$ and vice versa. In fact, this intuition is confirmed by the Eckart–Young–Mirsky Theorem.

That said, to compress an image, we aim for compute an SVD of the image and choose the right number of singular values k to minimize the used space while maximizing the quality of the compressed image.

2.1 Compression results

In the following, we will report the results obtained from the compression of the Valentino Castle image, a brick wall image, and a randomly generated (noise) image. We have compressed each of the images in both colored (figure 1) and greyscaled (figure 2).

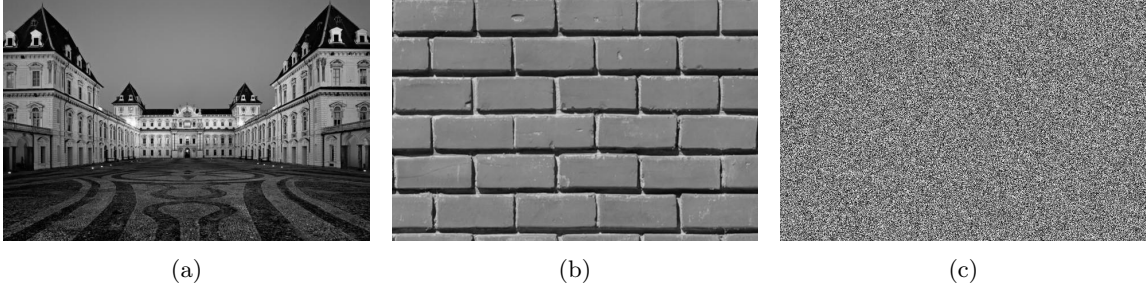


Figure 2: Grey-scaled images of our case study.

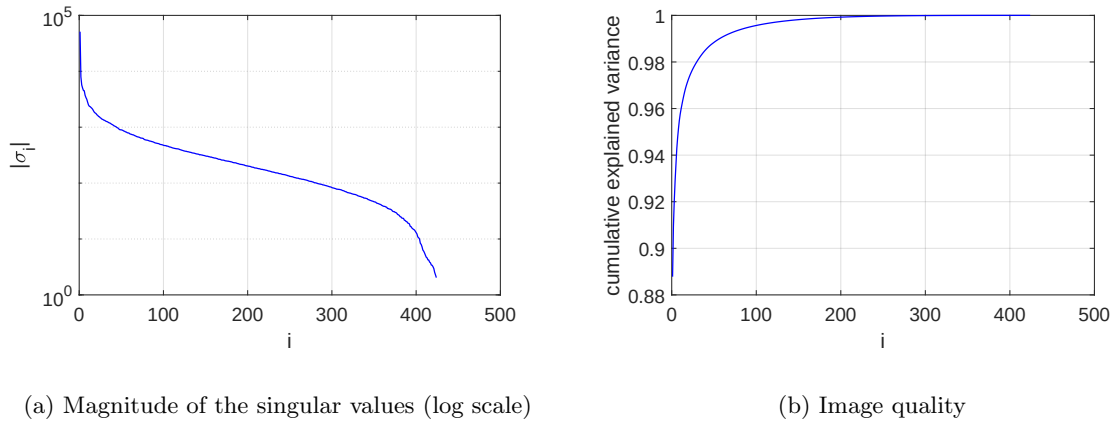


Figure 3: Magnitude of the singular values of the greyscaled Valentino image and their cumulative sum normalized in the range $(0, 1)$.

2.1.1 Valentino Castle

Figure 1a shows the image of Valentino Castle in Turin. First, we compress greyscaled image (figure 2a).

In figure 3a we can see the magnitude of all the singular values. An optimal way to choose the best k is to follow the elbow rule. We are looking for k such that adding $k + 1$ singular values does not significantly improve the quality of the compressed image, balancing this fact with the storage space.

We decided that a good trade off is achieved selecting $k = 25$. Figure 4 puts on view the compressed image as k varies; obviously, increasing k will increase the quality of the image.

It is interesting to see, even with just a singular value, we can clearly notice the brightness pattern of the image. As k increase, the details of the building emerge, even with relatively few singular values. At $k = 25$ most of the main details of the original figure are distinguishable: we can discriminate the sky, the building and the paving. Furthermore, the paving pattern is close to that in the original images and all the windows are well defined along with the less noticeable lightness.

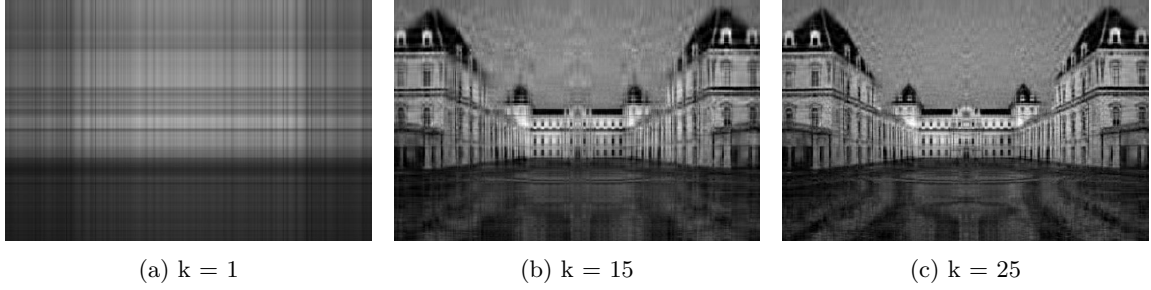


Figure 4: Compressed grey-scaled Valentino Castle's image

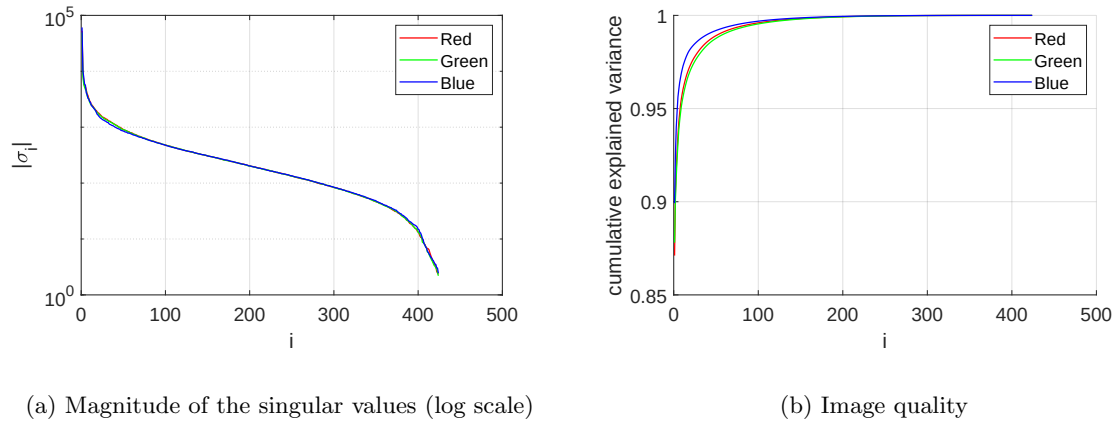


Figure 5: Magnitude of the singular values of the Valentino image and their cumulative sum normalized in the range $(0, 1)$.

In figure 5a, we present the magnitude of all singular values for the color image broken down into the three channel (red, blue and green). Looking at figure 5 we notice that the blue channel performs slightly better, this could be due to the larger presence of the color in the sky and in the paving.

We use the same rule to choose the optimal k . After the evaluation, we identified $k = 50$ as an effective trade-off. Figure 6 shows the compressed image with respect to k . As expected, higher k values correspond to better image quality. Interestingly, even with $k = 1$, we can perceive dominant color patterns across the image, capturing the overall brightness pattern. As k increases, finer details such as edges, color transitions, and textures emerge. For $k = 25$, the reconstructed image closely resembles the original: the contrast between the sky, the building, and the paving is well defined, and the details in the paving and building features, like windows and shadows, are clearly visible.

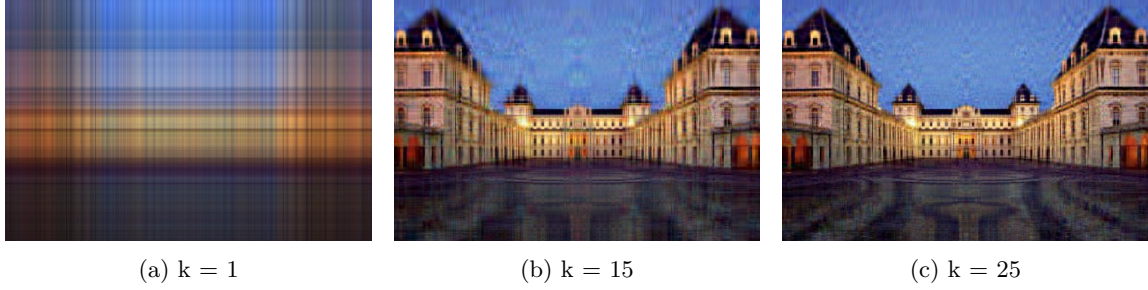


Figure 6: Compressed Valentino Castle's image

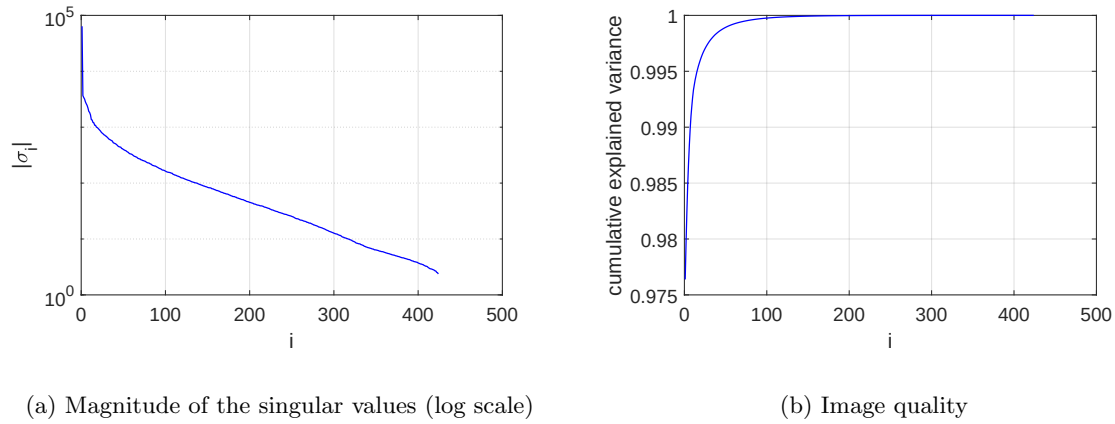


Figure 7: Magnitude of the singular values of the greyscaled brick wall image and their cumulative sum normalized in the range (0, 1).

2.1.2 Brick Wall

Figure 1b shows the image of a brick wall. We would find a correlation between images with pattern and enhancement using SVD compression. First, we analyze greyscaled image (figure 2b).

In figure 7a, we present the magnitude of all singular values for the greyscaled image. Compared to the Valentino Castle case, the firsts singular values in this image display higher magnitudes. It is also greater the cumulative explained variance (figure 7b). **TODO: [review: For instance, to achieve a cumulative ratio of 80%, the Valentino Castle image requires $k = 100$, while for the brick wall image a rank of $k = 50$ is enough to achieve the same result.]** As shown in figure 8a, even with a rank of $k = 1$ we can start to capture the rectangular pattern of the brick wall. At $k = 10$, the shape of the bricks is as it is in the original photo, in general, we achieved a good compression. Finally, for $k = 50$ are highly noticeable also the imperfection of the bricks.

In figure 9a, we present the magnitude of all singular values for the color image broken down into the RGB channel. Looking at figures 9 we notice that red channel outperforms the other two in both terms magnitude of first singular values and cumulative explained variance ratio. Just taking the largest singular value, we would achieve the 50% with respect to the total sum of the singular

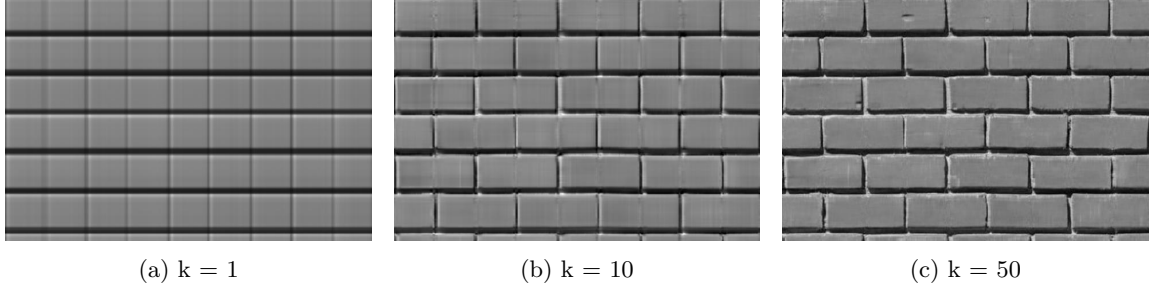


Figure 8: Compressed grey-scaled brick wall image

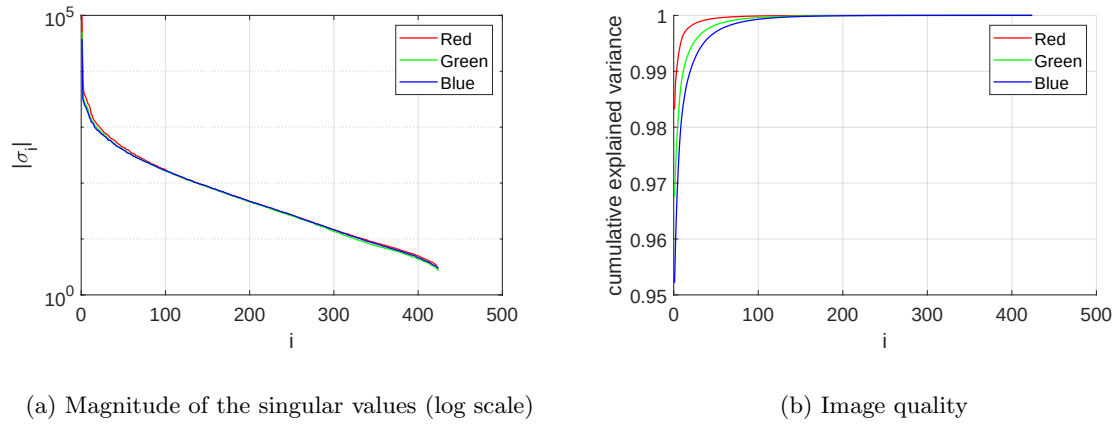


Figure 9: Magnitude of the singular values of the brick wall image and their cumulative sum normalized in the range (0, 1).

values. We could expect the result since the image has strongly oriented to the orange color.

We use the same rule to choose the optimal k . After the evaluation, we identified $k = 25$ as an optimal trade-off. Figure 10 shows the compressed image with respect to k . Interestingly, even with $k = 1$, we can perceive both almost the right shape of the bricks and dominant color pattern across the image. As k increases, finer details such as bricks' imperfection and textures emerge. At $k = 10$ the compressed image shows almost the right shape of the bricks without showing any imperfection of them. For $k = 25$, the reconstructed image closely resembles the original: the shadows and the colors within the bricks, the right color and the imperfection of the wall such as the not perfect shape and deterioration of the bricks.

2.1.3 Noise

The final image, shown in Figure 1c, is randomly generated using the implemented function `noise_image_generator`. In this case, we expect that the magnitude of the singular values will not vary significantly across the decomposition. This is because, unlike structured images, a random noise image lacks of dominant features or coherent structures that typically concentrate variance in

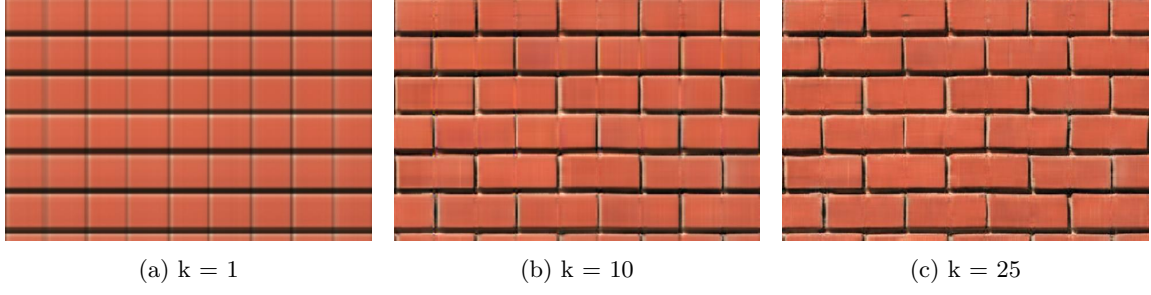


Figure 10: Compressed bricks wall's image

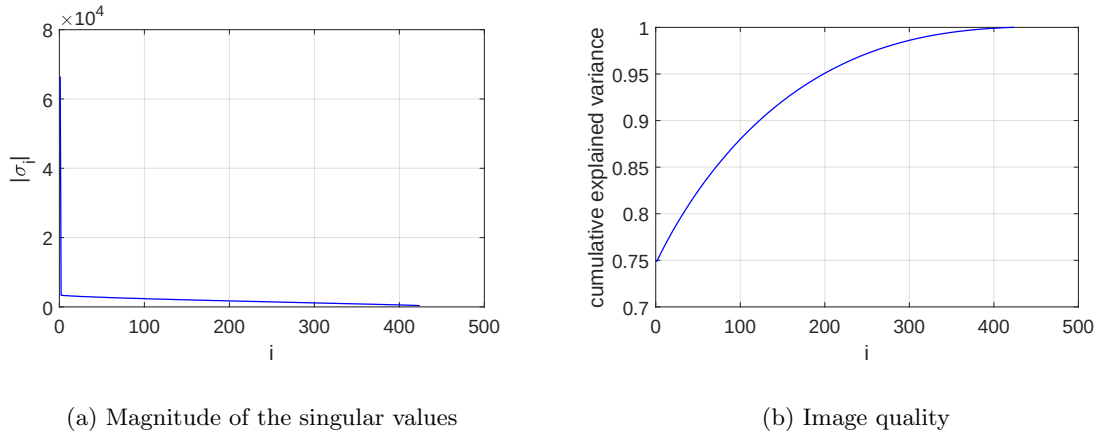


Figure 11: Magnitude of the singular values of the noise image and their cumulative sum normalized in the range $(0, 1)$.

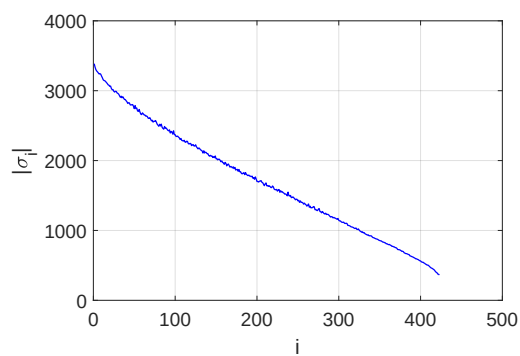
the largest singular values. On the contrary, looking at figure 11a we observe a steep initial rise in the curve. Analyzing manually the values of the first eigenvalues and looking at figure 11b where we can notice that it almost follows a linear trend but has an high value in zero; we understand that except for the first singular value, we achieved the thought result.

Figure 12a shows that if we exclude the first singular value, we would get a linear trend of the magnitude of singular values.

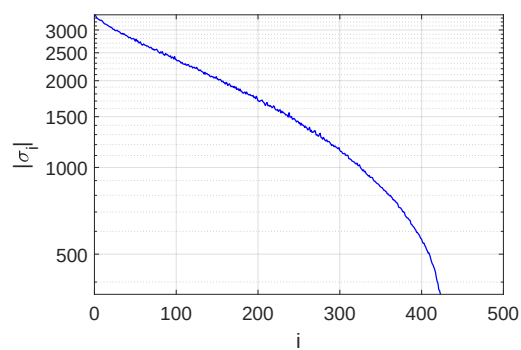
As discussed, we conclude that image compression using SVD is not useful to compress these kind of figures. Figure 13 shows the compressed images with respect to k , to achieve even a decent approximation of the original image a very high value of k is required, which leads to increased storage usage. We could figure out that the large magnitude of the first singular values is due to the pseudo-randomization of the `randi` function in Matlab as described in the documentation.

2.2 Effect of k on compression ratio and Mean Squared Error

In this section we show how the choice of k influences compression ratio and mean squared error of the compressed image with respect to the original image. To do so, some definitions need to be

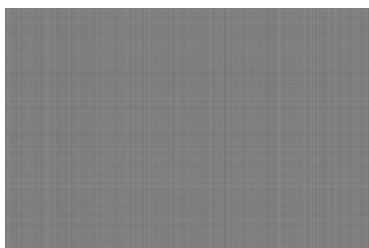


(a) Linear scale

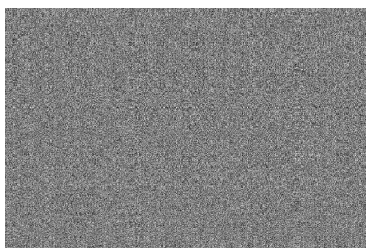


(b) Log scale

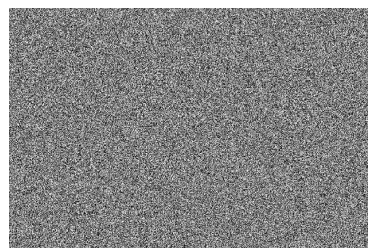
Figure 12: Magnitude of the singular values(except the first) of the noise image



(a) $k = 1$



(b) $k = 50$



(c) $k = 200$

Figure 13: Compressed noise image

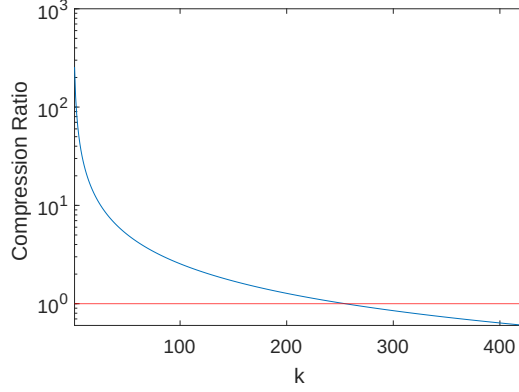


Figure 14: Compression ratio as k varies for an image whose size is 424×640 . The horizontal red line corresponds to a unitary compression ratio.

introduced first.

2.2.1 Compression Ratio as k varies

Compression ratio is defined as the ratio between original size of the image and size of the compressed image. The compression ratio of a picture of size $m \times n$ compressed with SVD storing k singular values and the corresponding singular vectors is the following.

$$C_R = \frac{m \cdot n}{k(m + n + 1)}$$

The denominator expresses the size of the compressed image using SVD: the cost of storing a singular vector consists in m elements for the left singular vector, n elements for the right singular vector and 1 element for the singular value.

Notice that compression ratio only depends on the size of the image being compressed and on k : it does not depend on the specific image. Figure 14 shows the compression ratio of an image whose size is the same as in the previously considered images as k varies. Compression ratio greater than 1 means that it's convenient to compress an image with that given k from the memory point of view: this means that it's convenient to compress an image whose size is 640×424 only up to $k = 254$.

2.2.2 Mean Squared Error as k varies

Mean Squared Error for a picture whose size is $m \times n$ is defined as follows

$$MSE = \frac{1}{mn} \sum_{y=1}^m \sum_{x=1}^n (f_A(x, y) - f_{A_k}(x, y))^2$$

where $f_A(x, y)$ is the value of the pixel in position (x, y) in the original image, while $f_{A_k}(x, y)$ is the value of the pixel in position (x, y) in the compressed image. The given definition is valid for

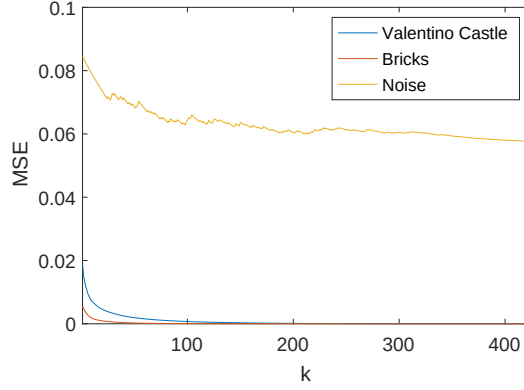


Figure 15: Mean squared error as k varies for previously considered images.

greyscale images, but naturally extends to images in RGB format computing MSE per channel or adding one dimension to f_A and f_{A_k} corresponding to the channel index. In the following analysis we consider the greyscale versions of the previously considered images. Figure 15 shows the mean squared error of the compressed images as k varies. Comparing the MSE of different images having fixed k lets us evaluate how a certain image is compressible with respect to the others.

- The Valentino castle's image is less compressible than the bricks image: this is due to the fact that the regular structure of the bricks image makes it easy to be reconstructed even if only few vectors are kept.
- The noise image is far less compressible than the other image: no regular patterns appear and MSE stagnates at 0.06 as k grows.

3 Compression Method Comparison

3.1 Discrete Cosine Transform

The Discrete Cosine Transform is a transformation that let images be represented as sinusoids of varying magnitudes and frequencies. Given a blocksize b , images of size $b \times b$ are decomposed as weights of b^2 basis functions organized into a $b \times b$ matrix. Components that contribute the most to the image are contained in the top-left corner of the matrix. To compress an image using DCT the following steps need to be performed.

1. The image is split into blocks of size $b \times b$, to each of this block the DCT is applied.
2. A mask is applied to each block so that only elements in the top left corner are kept.
3. To reconstruct the compressed image, inverse DCT is applied to each block.

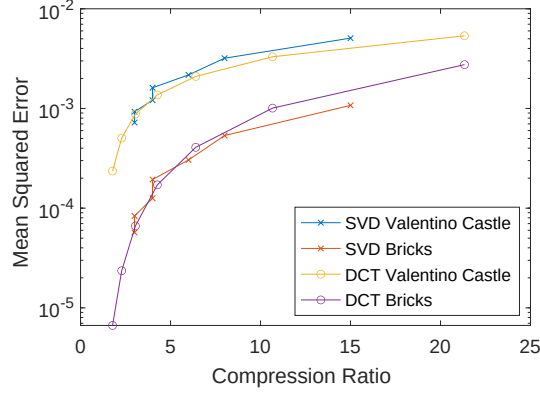


Figure 16: Scatter plot of Compression Ratio against Mean Squared Error for different compression schemes and different pictures. Lines joining points are shown to ease the comparison between methods and pictures.

If we apply a mask on the top left corner keeping $k < b$ elements in the first row or column, the number of nonzero elements is given by $k(k+1)/2$. For example in the following matrix

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

we have $k = 4$, $b = 8$ and 10 non-zero entries in the top left corner. Hence the compression ratio as a function of b and k is the following.

$$C_R = \frac{2b^2}{k \cdot (k+1)}$$

3.2 Comparison with DCT

In this section we choose $b = 8$ and consider the greyscale version of the Valentino castle image and the bricks image. We compress the image for different values of k using SVD and DCT:

- SVD is applied to images choosing $k = 3, 17, 31, 45, 58, 72, 86, 100$ where k is the number of singular values and singular vectors kept;
- DCT is applied to images choosing $k = 1, \dots, 8$ where k is the number of non-zero elements in the first row or column.

Figure 16 allows a comparison between SVD and DCT for the aforementioned pictures showing compression ratio versus mean squared error for the compressions described above.

- The Valentino castle image compression is better achieved with DCT rather than with SVD for high values of compression ratio.
- the bricks image compression is better achieved with SVD rather than with DCT for high values of compression ratio.

The results obtained are probably due to the fact that DCT in general performs better than SVD, but for images where there is a highly regular pattern that SVD can capture (as in the brick image), SVD leads to better results.